

UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

DEPARTMENT OF INFORMATION ENGINEERING

BACHELOR'S DEGREE IN COMPUTER ENGINEERING

Computing the median with limited memory: study and experimentation of the multi-pass algorithm by Munro and Paterson.

Supervisor

Prof. Andrea Pietracaprina

Candidate

Alessandro Barbacetto

ACADEMIC YEAR 2025–2026

Graduation date: November 2026

Note on this English version

This document is an English translation of the bachelor's thesis *Calcolo della mediana con memoria limitata: studio e sperimentazione dell'algoritmo multi-pass di Munro e Paterson*, originally written in Italian and discussed at the University of Padua.

The translation was produced with the help of an artificial intelligence tool and has **not been reviewed**. It is provided only to make the work accessible to non-Italian readers and may therefore contain inaccuracies, imprecise terminology or passages whose meaning differs slightly from the original. Figures are reproduced from the original version and may contain labels in Italian.

The reviewed and authoritative version is the original Italian one, which should be consulted for any reference or citation. It is available, together with the source code, in the public repository https://github.com/ill-barba/bsc-thesis-unipd-median-selection (file `thesis-alessandro-barbacetto.pdf`).

Contents

1 Introduction **1**

1.1 The selection problem and the median 1

1.2 Historical background 1

1.3 Goals and structure of the thesis 2

2 The median problem and the SELECT algorithm **3**

2.1 Problem definition and notation 3

2.2 Selection through a pivot 4

2.3 The SELECT algorithm: median of medians 6

2.4 Pseudocode and complexity 7

2.5 Limits of the classical model 8

3 Selection with limited memory **9**

3.1 Computational model and notation 9

3.2 General structure of the algorithm 10

3.3 Structure of a pass 11

3.3.1 Building the sample 12

3.3.2 Checking the memory constraints 14

3.3.3 Choosing the new pair of filters 15

3.4 Pseudocode of the algorithm 18

3.4.1 Building the sample 18

3.4.2 Single pass 20

3.4.3 Complete multi-pass algorithm 22

3.5 Correctness and termination 24

4 Implementation and experimental analysis **26**

4.1 Implementation choices 26

4.2 Implementation of the algorithm 27

4.3 Experimental methodology 28

4.4 Memory and number of passes 29

4.5 Actual candidates and theoretical bound 32

4.6 Final remarks and limitations 37

5 Application example **39**

5.1 Application scenario 39
5.2 Simplified model 41
5.3 Reference thermal window 42
5.4 Identifying the preparation laps 43
5.5 Possible extension to four tyres 44

Bibliography **47**

Acknowledgements **49**

Chapter 1

Introduction

1.1 The selection problem and the median

The selection problem consists in identifying, within a set or a sequence of elements drawn from a totally ordered domain, the element that would occupy a given position if the data were sorted. Given a rank k, which will be defined more precisely later, the goal is to determine the k-th element without necessarily resorting to a complete sorting of the input, which would be an immediate but unnecessary solution to the problem.

A particular case of the selection problem is the computation of the median. Once the input is sorted, the median corresponds to an element of central rank, and it can therefore be obtained as a special case of the problem of selecting an element of rank k.

As already mentioned, the selection of an element of rank k can be tackled without sorting the whole input beforehand, by exploiting the ability of selection algorithms to progressively discard the elements that cannot correspond to the solution being sought.

The goal of this thesis is therefore to start from these observations and focus on the case in which the amount of available working memory is very limited compared to the overall size of the input, studying how the median can be obtained through the multi-pass approach proposed by Munro and Paterson.

1.2 Historical background

The selection problem was already studied in the Seventies. A fundamental result was obtained in 1973 by Manuel Blum, Robert Floyd, Vaughan Pratt, Ronald Rivest and Robert Tarjan, who presented a deterministic algorithm, called *PICK*, able to select an element of given rank with a number of comparisons linear in the size of the input, even in the worst case [1]. This result laid the foundations of the approach that is nowadays commonly

presented through the *median of medians* technique, which will be discussed in the next chapter.

In 1980 J. Ian Munro and Michael S. Paterson instead addressed the selection problem by explicitly considering a limited amount of internal memory and an input that can only be accessed sequentially. In their paper *Selection and Sorting With Limited Storage* they focused in particular on the relation between the available memory and the number of passes required over the input, proposing a selection algorithm based on the recursive construction of samples organised on several levels [3]. This reference model and the multi-pass algorithm developed by the authors are the main subject of study of this thesis.

1.3 Goals and structure of the thesis

The main goal of this work is to study the problem of computing the median under limited memory, focusing on the multi-pass algorithm proposed by Munro and Paterson. The starting point is the classical selection problem and the deterministic *median of medians* algorithm, used as a reference to later introduce a model in which the whole input cannot be kept in memory at the same time, and where the need for a different method therefore arises.

After defining the computational model, a general overview of the Munro and Paterson algorithm will first be given, followed by a detailed discussion of the structure of a single pass, the construction of the samples, the handling of the memory bound and the choice of the filters, or sentinels, used to progressively narrow the set of candidates. The algorithm will finally be described through pseudocode, illustrating how it works, its correctness and its termination conditions.

The second part of the work is instead devoted to the actual implementation of the algorithm in a programming language and to its experimental evaluation. The experiments are carried out on datasets of different sizes, highlighting in particular the trade-off between the amount of available memory and the number of passes required over the input. The observed behaviour is then compared with the theoretical results, so as to obtain practical evidence of the trade-off analysed throughout the work. Finally, a possible application is presented in the "miniworld" of Formula 1, with the aim of showing how computing the median with limited memory can be used in a context characterised by sequential data acquisition and reduced memory availability.

Chapter 2

The median problem and the SELECT algorithm

2.1 Problem definition and notation

This section introduces the notation and the formal definitions used throughout the work, keeping a convention consistent with the one adopted by Munro and Paterson, which is then used again in the description of their algorithm [3]. Let:

$$A = (a_1, a_2, \ldots, a_N)$$

be a sequence of N elements belonging to a totally ordered domain, and assume that they are all distinct:

$$a_i \neq a_j \qquad \forall\, i, j \in \{1, \ldots, N\},\ i \neq j$$

No assumption is made on the order in which the elements appear in the input; if they were already sorted, the problem would be trivial. Let:

$$a_{(1)} > a_{(2)} > \ldots > a_{(N)}$$

denote the same collection of elements arranged in decreasing order.

Remark 2.1. The parentheses in the subscript distinguish the position of an element in the ordering from the one it occupies in the input sequence: $a_{(i)}$ is the element that would occupy position i if A were sorted, while a_i is the i-th element of the initial sequence.

Definition 2.1 (Rank)**.** The *rank* of a generic element $a_{(i)}$ is the position i it occupies in the sequence sorted in decreasing order. For example, $a_{(1)}$ is the largest element, $a_{(2)}$ the second largest and, in general, $a_{(k)}$ is the k-th largest element.

Definition 2.2 (Selection problem)**.** Given an integer k with $1 \leq k \leq N$, the *selection problem* consists in determining the element $a_{(k)}$ without sorting the whole sequence A.

Definition 2.3 (Median)**.** The *median* of A is the element $a_{(k)}$ obtained by solving the selection problem for:

$$k = \left\lceil \frac{N}{2} \right\rceil$$

that is, the element with the "central" index if N is odd, and the larger of the two "central" ones if N is even.

Without loss of generality, the decreasing order is adopted, from which the convention of the k-th largest element follows, in order to be aligned with the formulation of Munro and Paterson.

2.2 Selection through a pivot

A fundamental element of selection algorithms is the *pivot*, that is, an element of the sequence chosen as a reference to split the input. Once a pivot x is fixed, every other element is compared with it: since by assumption the elements are distinct, each element is classified as either greater or smaller than x. The pivot therefore occupies a uniquely determined position within the sorted sequence.

This mechanism makes it possible to establish in which of the two partitions the element of rank k lies, completely removing the other one from the search, so as to "thin out" the size of the problem and reduce it to a problem of the same kind defined on a smaller number of elements.

The choice of the pivot plays a decisive role in the efficiency of the algorithm, and there are various approaches to making this choice. One possible strategy is to select it at random, as happens in the randomised versions of QuickSort and QuickSelect. Such an approach is sometimes useful because it makes it unlikely to repeatedly obtain highly unbalanced partitions, and it generally leads to good performance in practice. The random choice, however, does not eliminate the worst case, it only tries to avoid it: if, for example, the selected pivot repeatedly falls close to one of the ends of the ordering, at each iteration only a very small subset of elements is removed, and the complexity can become quadratic in the size of the input [2].

In the problem of interest, however, a deterministic guarantee is required even in the worst case. It is therefore necessary to choose the pivot through a procedure that guarantees the removal of a sufficiently large fraction of the elements at each step, independently of their initial arrangement. The *SELECT* algorithm uses the *median of medians* technique

for this purpose: the pivot is not chosen arbitrarily or randomly, but is built so that it is not too close to the ends of the sorted sequence (the construction of the pivot and the guarantees provided by this choice will be discussed in the next section).

Once the pivot x has been chosen, its rank can be determined by counting how many elements of the sequence are greater than it. To do so, the elements of the input are split into two sets, separating those greater than the pivot from those smaller, denoting by:

$$G_x = \{a_i \in A \mid a_i > x\}$$

the set of elements greater than the pivot, and by:

$$L_x = \{a_i \in A \mid a_i < x\}$$

the set of elements smaller than the pivot.

Remark 2.2. Since by assumption all the elements of the sequence are distinct, no element can be at the same time greater and smaller than the pivot. The two sets are therefore disjoint and:

$$G_x \cap L_x = \varnothing$$

Setting $m = |G_x|$, in the decreasing order the pivot necessarily occupies position $m+1$. Comparing k with m, three mutually exclusive and exhaustive cases arise:

- if $k = m + 1$, the pivot coincides with the element being sought and the selection ends;
- if $k \leq m$, the k-th largest element necessarily belongs to G_x: the search continues only on this partition, keeping the rank k unchanged;
- if $k > m + 1$, the element being sought necessarily belongs to L_x. Since the m elements of G_x and the pivot itself are excluded from the search, the rank to look for in the new partition is "shifted" by an offset:

$$k' = k - (m + 1)$$

The selection is then applied recursively only to the partition containing the desired element, permanently discarding all the other elements. It can be observed that the quality of the obtained split, and therefore the number of elements that can be removed at each step, strictly depends on the choice of the pivot.

2.3 The SELECT algorithm: median of medians

The *SELECT* algorithm solves the selection problem by building the pivot deterministically through the *median of medians* technique. The underlying idea is not to choose an element of the input directly as the pivot, but to derive it through a series of intermediate medians, so as to obtain a sufficiently balanced split of the sequence [1, 2].

Given a sequence of N elements, it is first divided into groups of five elements each, with at most one last group containing fewer than five elements (without padding). This yields:

$$\left\lceil \frac{N}{5} \right\rceil \quad \text{groups} \quad G_1, G_2, \ldots, G_{\lceil N/5 \rceil}$$

The elements of each group G_i, with $1 \leq i \leq \lceil \frac{N}{5} \rceil$, are sorted and their median $m_i = \text{median}(G_i)$ is identified. In this way a new sequence made of the medians of the individual groups is obtained:

$$M = (m_1, m_2, \ldots, m_{\lceil N/5 \rceil})$$

The same *SELECT* algorithm is then applied recursively to this new sequence, with the goal of determining its median, that is:

$$x = \textsc{Select}\left(M, \left\lceil \frac{|M|}{2} \right\rceil\right)$$

The element x obtained in this way is called the *median of medians* and is promoted as the $pivot$ for the partition of the original input.

Once x is fixed, the original sequence is split into the sets G_x and L_x, as defined above. At this point, by comparing the target rank k with the rank of the pivot, it is possible to establish whether x coincides with the goal or on which of the two partitions the search should recursively continue, applying the same three cases seen before. The other partition can be permanently discarded.

The peculiarity of the algorithm therefore lies in the way the pivot is chosen. Because of the way x is chosen, it cannot lie arbitrarily close to the ends of the ordering. The structure of the groups of five indeed ensures that a sufficiently large number of elements is greater than x and, at the same time, that a sufficiently large number is smaller than x. This property prevents the recursive partition from keeping almost all the elements of the input and makes it possible to obtain a guarantee on the running time even in the worst case.

The quantitative guarantee provided by the choice of the median of medians will be presented in the next section, together with the time complexity of the algorithm.

2.4 Pseudocode and complexity

The operations described in the previous sections can be formalised in the following pseudocode:

Algorithm 1 SELECT

```
1:  function SELECT(A, k)
2:      N ← |A|
[B]
3:      if N ≤ 5 then
4:          *sort A in decreasing order*
5:          return A[k]
6:      *split A into ⌈N/5⌉ groups G_1, ..., G_⌈N/5⌉*
7:      for i ← 1 to ⌈N/5⌉ do
8:          *sort G_i in decreasing order*
9:          m_i ← G_i [⌈|G_i|/2⌉]
10:     M ← (m_1, m_2, ..., m_⌈N/5⌉)
[R]
11:     x ← SELECT(M, ⌈|M|/2⌉)
12:     G_x ← {a ∈ A | a > x}
13:     L_x ← {a ∈ A | a < x}
14:     m ← |G_x|
15:     if k = m + 1 then
16:         return x
17:     else if k ≤ m then
[R]
18:         return SELECT(G_x, k)
19:     else
[R]
20:         return SELECT(L_x, k − (m + 1))
```

The labels indicate, respectively: **[B]** the base case and **[R]** the recursive call to the SELECT algorithm.

The worst-case running time of *SELECT* can be expressed through the following recurrence:

$$T(N) \leq \begin{cases} \Theta(1) & N \leq 80 \\ T\left(\lceil \frac{N}{5} \rceil\right) + T\left(\frac{7N}{10} + 6\right) + O(N) & N > 80 \end{cases}$$

Here the first term is due to the selection of the median within the sequence M, while the second represents the maximum size of the partition on which the search continues, in the worst case. Finally, the term $O(N)$ is an upper bound on the operations needed

to build the groups, find their medians and partition with respect to the pivot. Solving the recurrence with the *unfolding* technique, that is, by repeatedly expanding its implicit definition, one obtains:

$$T(N) = O(N)$$

The *SELECT* algorithm therefore makes it possible to solve the selection problem deterministically in linear time, even in the worst case [1, 2].

2.5 Limits of the classical model

The *SELECT* algorithm shows that the selection problem can be solved deterministically in linear time. This result, however, implicitly assumes a data access model consistent with the RAM machine: the whole input must reside in main memory, so that its elements can be freely accessed, repeatedly partitioned, and the subsequences involved in the subsequent recursive calls can be kept.

Note that this assumption no longer holds when the size of the input is much larger than the available working memory. In that case it is no longer possible to keep all the elements of the input in memory at the same time, nor to directly apply the partitioning operations on which *SELECT* relies. Running time alone is no longer a consistent cost model for the problem; consequently, it becomes necessary to also take into account the amount of available memory (on the device on which the code runs) and the ways in which the data can be accessed.

The study by Munro and Paterson focuses precisely on this scenario, considering an input that can only be accessed sequentially, like a one-way read tape, and an internal memory of small size compared to the input. Under these assumptions, the number of passes over the input becomes a computational resource on a par with working memory, and rereading the sequence several times makes it possible to obtain, pass after pass, the information that in the classical model would be obtained with a single random access to the data, at the cost, however, of a longer overall running time [3]. The next chapter will formally introduce this computational model and show how the selection problem can be reformulated under such constraints.

Chapter 3

Selection with limited memory

3.1 Computational model and notation

This section introduces the new quantities needed to extend the model defined in the previous chapters and to describe the selection algorithm with limited memory in more detail. We start from:

Definition 3.1 (Input tape)**.** The sequence A is stored on a reading device, called a *tape*, which allows the elements $a_1, a_2, \ldots, a_N$ to be read only sequentially and in one direction, in the order in which they are stored. The elements on the tape cannot be modified and, once a given position has been passed, it is not possible to go back to it without first bringing the tape back to the initial position.

Definition 3.2 (Available memory)**.** Let $S \in \mathbb{N}$ be the number of random-access memory cells available to the algorithm. An element read from the tape can be transferred into one of the available cells, and two elements can be compared only if both are in memory at the same time.

Definition 3.3 (Pass)**.** A *pass* is a complete scan of the tape, from the initial position a_1 to the final position a_N. At the end of a pass the tape is brought back to the initial position, without performing further reads in the meantime, so that a possible new pass can begin.

Definition 3.4 (Number of passes)**.** Let $P \in \mathbb{N}$ be the total number of passes performed over the tape before the algorithm returns the element of rank k.

To progressively narrow the part of the ordering in which the element being sought can lie, the algorithm keeps in memory a pair of elements, called *filters*.

Definition 3.5 (Filters)**.** The *filters* are two values $f_{\min}$ and $f_{\max}$ kept in memory during the execution, such that:

$$f_{\min} \leq a_{(k)} \leq f_{\max}$$

The filters therefore delimit an interval of the ordering that always contains the element being sought, acting as sentinels. At the beginning of the algorithm, since no information on the position of $a_{(k)}$ is available yet, they are initialised by default to $-\infty$ and $+\infty$.

Definition 3.6 (Candidates)**.** Given a pair of filters $f_{\min}$ and $f_{\max}$, the set of *candidates* of the pass can be defined as:

$$C = \{a_i \in A \mid f_{\min} < a_i < f_{\max}\}$$

and $n = |C|$ denotes the cardinality of the set, that is, the number of elements strictly between the two filters.

Remark 3.1. The elements of C are those that, at the current state, cannot yet be excluded from the search. The two filters do not belong to C and are kept separately in memory, since one of them might already coincide with $a_{(k)}$.

Note also that the subscript of the candidate elements is not enclosed in parentheses: the elements of C, denoted a_i, still keep the position they occupied in the sequence on the tape.

The selection problem therefore reduces to determining $a_{(k)}$ using at most S memory cells and P passes over the tape. Moreover, the following invariant can be stated for the algorithm.

Invariant 3.1 (Filter invariant)**.** Let $f_{\min}^{(p)}$ and $f_{\max}^{(p)}$ be the filters used at the beginning of a generic pass p. For every pass performed by the algorithm the following must hold:

$$f_{\min}^{(p)} \leq a_{(k)} \leq f_{\max}^{(p)}$$

consequently, the element of rank k is never excluded from the search interval during the execution of the algorithm.

3.2 General structure of the algorithm

The Munro and Paterson algorithm replaces the direct partitioning of SELECT with a progressive reduction of the interval identified by the filters. The algorithm that follows

is the one proposed by the authors for the case in which the memory is sufficiently large, namely:

$$S \in \Omega((\log N)^2)$$

At the beginning, the filters are initialised by default to $-\infty$ and $+\infty$, so $n = N$ and all the elements of the input are considered candidates. The algorithm then performs a sequence of complete passes over the tape. During each pass two things happen at the same time: the information needed to determine the exact rank of the current filters is collected, and a sample that can "fit" the available memory is built from the remaining candidates.

The sample does not contain all the candidates, but only a subset, chosen so as to remain representative of the original distribution. From this sample, two new filters are then obtained, closer to each other than the previous ones but still such that they contain $a_{(k)}$. At this point the tape is rewound and a new pass is performed using the filters just computed; n thus decreases from pass to pass, without it ever being necessary to keep the whole sequence of candidates in memory.

The algorithm stops sampling when the upper bound on the number of candidates drops to $S - 2$ or less; at this point the base case is reached and the remaining elements can be read and kept all at the same time in memory (together with the two filters), and the selection ends with a last direct pass over the few remaining elements. Note that the execution may also end earlier, for example if, while computing the rank of a filter, it turns out that the filter coincides exactly with $a_{(k)}$.

The three ingredients to be defined are therefore: how the sample of a pass is built, how to verify that the S available cells are never exceeded, and how the new pair of filters is obtained from this sample. These steps will be described and implemented in detail in the next sections.

3.3 Structure of a pass

Consider a pass that starts with filters $f_{\min}$, $f_{\max}$ and with n candidates. The tape must be read entirely anyway, because there is no way to access only the positions containing elements of C: every element read is compared with the current filters.

The elements that fall outside the interval $(f_{\min}, f_{\max})$ do not enter the sample, but they are needed to update the counts used to compute the rank of the filters. The elements that instead fall inside the interval, that is, the candidates, form the population from which the sample of the pass is built. Once the scan is over, the sample obtained is used to choose the filters for the next pass.

3.3.1 Building the sample

The sample must reconcile two constraints pulling in opposite directions: it must occupy much less space than the n starting candidates, but still retain enough information about their distribution in the ordering to narrow the search interval safely. Munro and Paterson resolve this trade-off with a recursive construction, organised on several levels [3], defined as follows:

Definition 3.7 (Sample of level i)**.** Let s be a positive even integer. A *sample of level i* is a sorted sequence of s elements, obtained from a population of $2^i s$ candidate elements according to the recursive construction:

- **Base case ($i = 0$)**

  When $i = 0$ the population reduces to $2^0 s = s$ elements; the sample of level 0 then coincides with the whole population, simply sorted in decreasing order. At this level there is still no loss of information, because nothing has been discarded.

- **Recursive step ($i + 1$)**

  To build a sample of level $i+1$ one starts instead from a larger population of $2^{i+1} s$ elements, which is divided into two consecutive halves of $2^i s$ elements each, following the order in which the candidates are encountered on the tape.

  From each half a sample of level i is obtained recursively, thus producing two sorted samples of s elements each. These two samples, however, cannot simply be joined, because this would produce a sequence of $2s$ elements, while a sample must always keep s of them. Before combining them it is therefore necessary to reduce them through the *thinning* operation.

Definition 3.8 (Thinning)**.** Let:

$$X = (x_1, x_2, \ldots, x_s)$$

be a sample of s elements, with s even, sorted in decreasing order. The *thinning* operation consists in keeping one element out of every two, that is, keeping only those occupying an even position:

$$\text{thin}(X) = (x_2, x_4, x_6, \ldots, x_s)$$

The size of the sample is therefore reduced from s to $s/2$ elements.

Applied to the two samples of level i, thinning reduces them to $s/2$ elements each, and at this point they can be recombined with a *merge*, going back to a sorted sequence of s elements in total. The overall result is the desired sample of level $i + 1$.

The whole procedure is easy to understand when visualised as a binary tree (from the bottom up), shown in Figure 3.1, where the leaves correspond to the samples of level 0, each internal node applies thinning to the two subsamples of its children and merges them, and the root of the tree, reached at the highest level r, is the final sample used to update the filters.

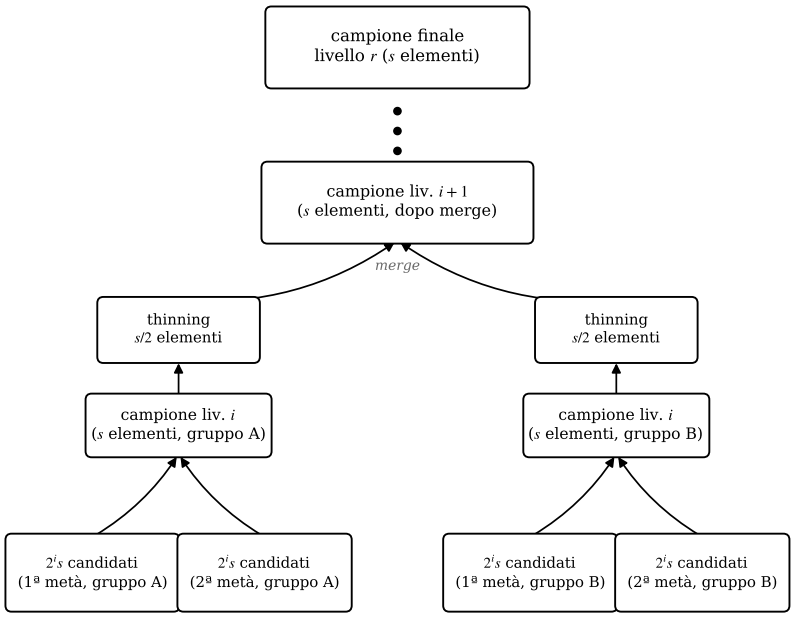


Figure 3.1: Recursive construction of the sample: at each level the two subsamples are "thinned" and then joined with a merge, until the final sample of level r is reached.

Remark 3.2. Although it is organised on r levels, the construction of the sample does not require one pass per level: all levels from 0 to r are built during a single scan of the tape. The tree in Figure 3.1 indeed represents only the *logical* dependencies between the subsamples, which do not all have to be kept in memory at the same time.

To understand how memory occupation varies during the recursion, one can follow the construction of two consecutive subsamples. At level 0 a working buffer fills up progressively, one candidate at a time, until it contains s elements. The sample obtained in this way is sorted and, when it has to be combined with another sample, reduced through *thinning* to $s/2$ elements. This first half is kept temporarily, while the same working buffer is reused to build the second one. Once the second half is also available, the *merge* produces

a new sample of s elements, reusing the same working space. At that point the first half waiting is no longer needed and its cells become available again.

The procedure is repeated at the higher levels as well: each intermediate level can therefore have a waiting slot that is either empty or occupied by a single, already thinned subsample of $s/2$ elements. Two complete samples of s elements per level never accumulate, because the first one is reduced before the second one is built. In this way, in addition to the shared buffer of s elements and to the two cells of the filters, at most $s/2$ cells are needed for each of the r intermediate levels [3, 4].

It remains to establish at which level r it is convenient to stop this construction. At the beginning of the pass the exact number of candidates is not yet known, but only an upper bound $\bar{n}$ on it; one therefore chooses the smallest r whose population of $2^r s$ elements is able to contain, even in the worst case, all the expected candidates. It must therefore hold that:

$$\bar{n} \leq 2^r s$$

from which, solving for r:

$$r = \left\lceil \log_2 \left(\frac{\bar{n}}{s} \right) \right\rceil$$

The sample of level r obtained in this way is the final sample returned by the pass (the root of the tree shown).

If the actual number of candidates is strictly smaller than $2^r s$ (which happens most of the time, unless $\bar{n}$ is exactly a power of two times s), the population is completed with dummy elements (through a *padding* operation), in order to reach the size required by the recursive structure.

3.3.2 Checking the memory constraints

Of course, this construction is valid only if it fits within the S memory cells actually available, and it is worth verifying this before going further. As described in the previous section, the recursive construction reuses a single working buffer of s elements and keeps, for each of the r intermediate levels, at most one already thinned subsample of $s/2$ elements waiting to be combined with the next one.

The total space required is therefore: that of one subsample for each of the r levels, plus a working sample of s elements, plus the two cells occupied by the filters, in total:

$$\frac{rs}{2} + s + 2 \leq S$$

This constraint ties s and r together, which therefore cannot be chosen independently: for example, a larger sample takes up more space, but also allows stopping at a lower level r. Munro and Paterson resolve the trade-off by setting:

$$s = 2\left\lfloor \frac{S}{2\log_2 \bar{n}} \right\rfloor \qquad r = \left\lceil \log_2 \left(\frac{\bar{n}}{s} \right) \right\rceil$$

a choice that keeps s even and satisfies the constraint for sufficiently large S [3]. The resulting behaviour is fairly intuitive: if s grows proportionally to $S/\log \bar{n}$ (more memory is available), the sample can be larger; with little memory, on the contrary, the sample necessarily remains small and more levels are needed, that is, a larger r.

3.3.3 Choosing the new pair of filters

At this point the sample is ready, but it remains to understand how to use it to narrow the search interval without risking, in doing so, to lose $a_{(k)}$. The procedure relies on the idea that each element of the sample occupies, within the population from which it was extracted, a position that is not known exactly, but that is nonetheless enclosed in an interval whose endpoints are known. Before formalising it, however, it is convenient to fix the quantities involved.

Definition 3.9 (Rank bounds of the sample)**.** For the j-th largest element of a sample of level i, L_{ij} and M_{ij} denote, respectively, the minimum and the maximum number of elements of the starting population that can lie above it in the ordering. Munro and Paterson prove that these bounds are explicitly given by:

$$L_{ij} = j2^i - 1, \qquad M_{ij} = (i + j - 1)2^i$$

In other words, it is not known exactly how many elements of the population precede a given element of the sample, but it is known that they are no more than M_{ij} and no fewer than L_{ij} [3]. It is this "double guarantee" that makes it possible to use the sample to narrow the search interval without making errors.

Part (a) of Figure 3.2 gives a visual representation of the guarantee on the "rank line": since L_{ij} and M_{ij} count the elements preceding x_j, the actual rank of x_j within the population lies between $L_{ij} + 1$ and $M_{ij} + 1$. The element x_j therefore does not have a known position, but it is nonetheless constrained not to fall outside this interval. To compare L_{ij} and M_{ij} with the rank k of the element being sought, however, there is an obstacle to clarify: k is defined with respect to the whole sequence A, while L_{ij} and M_{ij} describe positions *within the current population* of the candidates only. The two quantities therefore

refer to different scales and, consequently, are not yet directly comparable.

To bring them to the same scale, the rank of the filters is needed, and it is obtained precisely during the scan of the tape.

Definition 3.10 (Rank of a filter)**.** For a real filter x, recalling that the elements of A are all distinct, its rank is defined as:

$$\rho(x) = 1 + |\{a_i \in A \mid a_i > x\}|$$

For the ideal (initial) filter, by convention $\rho(+\infty) = 0$, consistently with the fact that no element of the sequence can exceed it.

The value $\rho(f_{\max})$ is obtained simply by counting, while reading the tape, how many elements are greater than $f_{\max}$. Since all the candidates occupy, by construction, the positions immediately following $f_{\max}$ in the ordering of A, subtracting $\rho(f_{\max})$ from the rank k gives exactly the rank that $a_{(k)}$ would have if only the candidates of the current population were counted, starting from $f_{\max}$.

Definition 3.11 (Relative rank)**.** The *relative rank* of $a_{(k)}$ with respect to the current population of candidates is the quantity:

$$k_C = k - \rho(f_{\max})$$

The step from k to k_C therefore does not introduce any new unknown: k_C is still the rank of the element being sought, only measured from the beginning of the current population instead of from the beginning of the whole sequence. This is precisely the common convention that makes it possible to compare k_C with the bounds L_{ij} and M_{ij} of Definition 3.9.

After this conversion, the new filters can be chosen directly from the bounds of Definition 3.9. Denoting by $x_1 > x_2 > \ldots > x_s$ the final sample of level r, for the upper filter one looks for the largest index u such that, even in the worst case, no more than $k_C - 1$ elements can lie above x_u. One therefore looks for:

$$M_{ru} = (r + u - 1)2^r \leq k_C - 1$$

from which, solving for u:

$$u = \left\lceil \frac{k_C}{2^r} \right\rceil - r$$

Similarly, for the lower filter one looks instead for the smallest index v such that at least

$k_C - 1$ elements certainly lie above x_v. It must therefore hold that:

$$L_{rv} = v2^r - 1 \geq k_C - 1$$

from which, solving for v:

$$v = \left\lceil \frac{k_C}{2^r} \right\rceil$$

Finally, one sets $f'_{\max} = x_u$ and $f'_{\min} = x_v$ and, by construction, $a_{(k)}$ must lie between the two new filters or coincide with one of them. Invariant 3.1 therefore continues to hold after the update, thanks to the conversion performed earlier.

Part (b) of Figure 3.2 shows this conclusion on the same "rank line" used above for (a), where the indices u and v identify the elements x_u and x_v of the final sample, which become the new filters $f'_{\max}$ and $f'_{\min}$. The band highlighted in black between x_u and x_v is the interval within which $a_{(k)}$ is guaranteed to lie, based on how u and v were chosen from Definition 3.9.

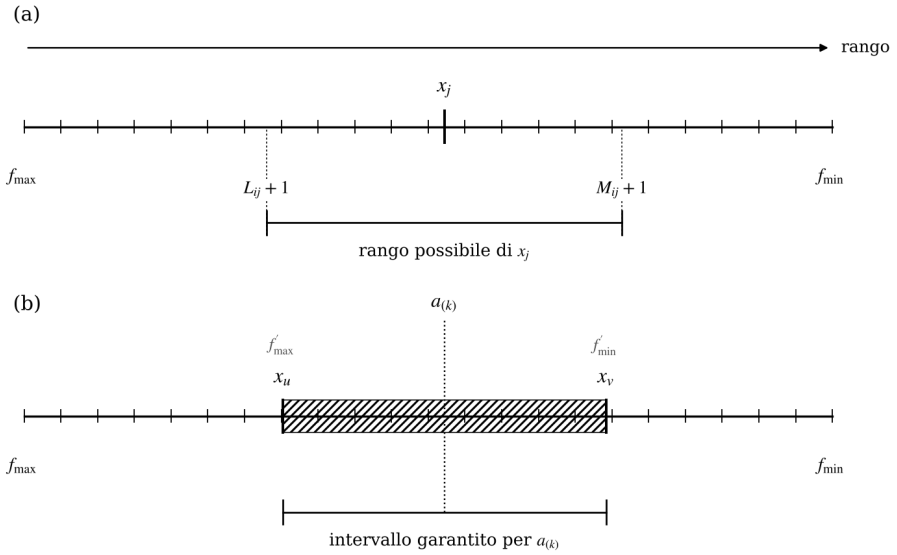


Figure 3.2: Graphical interpretation of the quantities introduced in this section. In panel (a), $L_{ij} + 1$ and $M_{ij} + 1$ are the endpoints of the interval in which the rank of a generic element x_j of the sample can lie. In panel (b), the indices u and v identify the elements x_u and x_v of the final sample of level r that become the new filters $f'_{\max}$ and $f'_{\min}$.

It only remains to define what the actual gain obtained with the pass just completed has been, that is, how many candidates can still lie between the new filters x_u and x_v. Applying Definition 3.9 again, but to the indices u and v, the maximum number of elements between the two turns out to be:

$$M_{rv} - L_{ru} - 1 = (2r - 1)2^r$$

substituting the values of r and s fixed in Section 3.3.2, one finds:

$$(2r-1)2^r \in O\left(\frac{\bar{n}(\log \bar{n})^2}{S}\right)$$

A single pass therefore reduces the upper bound on the number of candidates from $\bar{n}$ to $O(\bar{n}(\log \bar{n})^2/S)$, and it is precisely this reduction, obtained pass after pass by deterministically repeating the same procedure, that makes the algorithm converge towards a population small enough to be held entirely in memory, in a single last pass.

3.4 Pseudocode of the algorithm

The ideas presented in the previous sections are the theory needed to formalise a pseudocode for this algorithm.

Before writing the code, however, it is worth clarifying a point that has so far been left in the background and was anticipated in Section 3.3.1: at the beginning of a pass the exact number n of candidates is not yet known, because knowing how many elements fall between the filters would require examining the tape. To fix the parameters s and r of the sample construction it is therefore enough to know an upper bound on this number, denoted $\bar{n}$. In the first pass this bound is simply $\bar{n} = N$, while in the subsequent passes the bound derived in Section 3.3.3 directly provides the value to be used for the next one.

Remark 3.3. The introduction of $\bar{n}$ therefore does not require an additional pass devoted solely to counting the candidates: it is an upper bound already known before the scan begins. On the other hand, $n = |C|$ remains, as before, the actual number of candidates observed during the pass.

3.4.1 Building the sample

The pseudocode for building the sample derives from Definition 3.7 and is, essentially, a direct translation of the recursive construction described in Section 3.3.1. To make the reading more "operational", the primitive NextCand is introduced, which continues the scan from the current point until it meets the next candidate. It is PASS, not SAMPLE, that positions the tape at the beginning of the sequence and initialises NextCand, only once, at the start of the pass; the current filters $f_{\min}$ and $f_{\max}$ are fixed at that same moment. All the recursive calls of SAMPLE that follow, including the two calls at level $i-1$ in the recursive step, share the same NextCand and therefore the same read cursor: none of them rewinds the tape or initialises its own copy, and the stream of candidates is thus

traversed only once, one element at a time.

Every element x read is in any case compared with the filters and, if needed, used to update the counters employed in computing their rank. The procedure, instead, returns only the elements that satisfy $f_{\min} < x < f_{\max}$, that is, those belonging to the set C of candidates.

If the tape ends without further candidates being available, NextCand returns the "flag" End. In this case SAMPLE uses the dummy elements needed to complete the population required by the recursive construction, through a *padding* operation.

Algorithm 2 Recursive construction of the sample

```
 1: function SAMPLE(i, s)
[B]
 2:     if i = 0 then
 3:         X ← ∅
 4:         for j ← 1 to s do
 5:             x ← NextCand
 6:             if x = End then
 7:                 x ← *padding*
 8:             *insert x into X*
 9:         *sort X in decreasing order*
10:         return X
[R]
11:     X_1 ← SAMPLE(i − 1, s)
12:     X'_1 ← (X_1[2], X_1[4], ..., X_1[s])
[R]
13:     X_2 ← SAMPLE(i − 1, s)
14:     X'_2 ← (X_2[2], X_2[4], ..., X_2[s])
15:     return Merge(X'_1, X'_2)
```

In the base case the procedure reads a block of s candidates and sorts them, which corresponds exactly to the sample of level 0 of Definition 3.7. In the recursive case, instead, the two samples of level $i-1$ are built one after the other and each of them is reduced with *thinning*, and only at the end are the two subsamples combined through a *merge*.

Remark 3.4. The order of the operations is not accidental. The first sample must be thinned before building the second one, so as to keep in memory only $s/2$ waiting elements at each level of the recursion. In this way memory occupation remains consistent with the space constraint derived in Section 3.3.2. Note also that X'_1 and X'_2 are not additional copies kept next to X_1 and X_2: thinning is to be understood as performed directly in the original buffer, overwriting the starting sample with the $s/2$ selected elements, so that the two versions of the same subsample never appear in memory at the same time. The cells

that are no longer needed become available for the subsequent operations.

3.4.2 Single pass

A single pass actually has to carry out three tasks together: keep track of the rank of the current filters, build the sample from the candidates only, and finally choose the pair of filters to pass on to the next scan.

For the first task two counters are enough: one counts how many elements of the input exceed $f_{\max}$, while the other counts how many exceed $f_{\min}$. Once the end of the tape is reached, these two values immediately give the exact rank of the filters, without the need for a dedicated scan.

Algorithm 3 Single pass

function PASS($A, k, S, f_{\min}, f_{\max}, \bar{n}$)
  $s \leftarrow 2\left\lfloor \dfrac{S}{2\log_2 \bar{n}} \right\rfloor$
  $r \leftarrow \left\lceil \log_2\left(\dfrac{\bar{n}}{s}\right) \right\rceil$
  $q_{\max} \leftarrow 0$
  $q_{\min} \leftarrow 0$
  rewind the tape and initialise NextCand
  $X \leftarrow$ SAMPLE(r, s)
  complete the scan of the tape updating $q_{\max}$ and $q_{\min}$
  if $f_{\max} = +\infty$ **then**
    $\rho_{\max} \leftarrow 0$
  else
    $\rho_{\max} \leftarrow q_{\max} + 1$
  if $f_{\min} = -\infty$ **then**
    $\rho_{\min} \leftarrow N + 1$
  else
    $\rho_{\min} \leftarrow q_{\min} + 1$
  if $\rho_{\max} = k$ **then**
    return (Found, $f_{\max}$)
  if $\rho_{\min} = k$ **then**
    return (Found, $f_{\min}$)
  $k_C \leftarrow k - \rho_{\max}$
  $u \leftarrow \left\lceil \dfrac{k_C}{2^r} \right\rceil - r$
  $v \leftarrow \left\lceil \dfrac{k_C}{2^r} \right\rceil$
  $f'_{\max} \leftarrow X[u]$
  $f'_{\min} \leftarrow X[v]$
  $\bar{n}' \leftarrow (2r-1)2^r$
  return (Continue, $f'_{\min}, f'_{\max}, \bar{n}'$)

Remark 3.5. Note that the procedure PASS receives $\bar{n}$ as input and not the actual number n of candidates. The value $\bar{n}$ indeed represents the upper bound known before the beginning of the pass and is sufficient to determine the parameters s and r of the sample construction.

Moreover, the procedure can end by returning two different states:

- FOUND: indicates that one of the filters has rank exactly equal to k and therefore coincides with the element being sought. Together with the flag, the value of the filter is returned.
- CONTINUE: indicates that none of the filters coincides with $a_{(k)}$ and that the search must therefore continue. In this case the new filters $f'_{\min}$, $f'_{\max}$ and the upper bound $\bar{n}'$ to be used in the next pass are also returned.

During the pass two counters, $q_{\max}$ and $q_{\min}$, are maintained, which keep track respectively of the number of elements read from the tape that are greater than $f_{\max}$ and than $f_{\min}$. Both are updated during the same scan performed by NEXTCAND. Once the construction of the sample is complete, any elements still on the tape are simply read until the end to complete the two counts.

3.4.3 Complete multi-pass algorithm

The complete algorithm simply repeats the PASS procedure until the upper bound on the number of candidates drops to $S-2$ or less, at which point the problem can be reduced to the trivial solution. Indeed, once that threshold is reached, all the elements still between the filters can finally be kept in memory together with the filters themselves, and building a further sample is no longer necessary.

Before presenting the complete pseudocode, it is worth specifying how the result returned by PASS is used. This result is stored in the object R, whose field $status$ can take two values depending on the state returned by the procedure:

- If $R.status$ = FOUND, the field $R.value$ directly contains the element of rank k and the algorithm ends.
- If instead the state is CONTINUE, R contains the new filters $f_{\min}$, $f_{\max}$ and the new upper bound $\bar{n}$, which become the parameters of the next pass.

This procedure is executed while $\bar{n} > S-2$; since the actual number of candidates satisfies $n \leq \bar{n}$, when this condition no longer holds it is certainly true that $n \leq S-2$. At that point all the remaining candidates can be kept in memory at the same time together with the two filters, so it is no longer necessary to build a new sample and the selection can be completed with a last pass over the tape.

Algorithm 4 Multi-pass selection

```
1:  function MP-SELECT(A, k, S)
2:      f_min ← −∞
3:      f_max ← +∞
4:      n̄ ← N
5:      while n̄ > S − 2 do
6:          R ← PASS(A, k, S, f_min, f_max, n̄)
7:          if R.status = FOUND then
8:              return R.value
9:          f_min ← R.f_min
10:         f_max ← R.f_max
11:         n̄ ← R.n̄
12:     C ← ∅
13:     q_max ← 0
14:     q_min ← 0
15:     *rewind the tape*
16:     for each x ∈ A do
17:         if x > f_max then
18:             q_max ← q_max + 1
19:         if x > f_min then
20:             q_min ← q_min + 1
21:         if f_min < x < f_max then
22:             C ← C ∪ {x}
23:     if f_max = +∞ then
24:         ρ_max ← 0
25:     else
26:         ρ_max ← q_max + 1
27:     if f_min = −∞ then
28:         ρ_min ← N + 1
29:     else
30:         ρ_min ← q_min + 1
31:     if ρ_max = k then
32:         return f_max
33:     if ρ_min = k then
34:         return f_min
35:     k_C ← k − ρ_max
36:     *sort C in decreasing order*
37:     return C[k_C]
```

3.5 Correctness and termination

The correctness of the whole algorithm rests entirely on Invariant 3.1: if at the beginning of every pass the position of $a_{(k)}$ is guaranteed to lie between the two current filters, proving that the algorithm is correct reduces to three checks:

- that the invariant holds at the beginning of the execution
- that it continues to hold every time the filters are updated
- that the final procedure actually returns the element of rank k

The proof of these three points follows:

Initialisation: Initially the filters are set to $-\infty$ and $+\infty$, so trivially:

$$-\infty \leq a_{(k)} \leq +\infty$$

The invariant therefore holds even before the algorithm performs its first operation, whatever the input sequence A.

Maintenance: Suppose that the invariant holds at the beginning of a generic pass. If the rank of one of the two filters coincides exactly with k, PASS detects it in the same pass and returns the state FOUND together with that filter: the selection ends there, correctly.

In the complementary case, $a_{(k)}$ lies strictly between the two filters, and it is up to the sample of the pass to establish how to further narrow the interval, returning the state CONTINUE. Denoting by $\rho(f_{\max})$ the rank of the upper filter, the position of $a_{(k)}$ relative to the population of candidates only is, by Definition 3.11:

$$k_C = k - \rho(f_{\max})$$

Now let $X = (x_1, x_2, \ldots, x_s)$ be the final sample of level r built during the pass. By Definition 3.9, each of its elements x_j is bound to the limits L_{rj} and M_{rj}, and it is precisely these limits, applied to the indices u and v derived in Section 3.3.3, that guarantee:

$$M_{ru} \leq k_C - 1 \leq L_{rv} \qquad \text{hence} \qquad M_{ru} + 1 \leq k_C \leq L_{rv} + 1$$

The left-hand side of the inequality states that, even in the worst case, no more than $k_C - 1$ elements can lie above x_u; x_u therefore cannot be smaller than the element of relative rank k_C. The right-hand side states instead that at least $k_C - 1$ elements lie above x_v, so x_v cannot be larger than the element being sought. This is exactly the situation in part (b)

of Figure 3.2. Putting the two inequalities together, $a_{(k)}$ can be placed in the highlighted band between x_v and x_u:

$$x_v \leq a_{(k)} \leq x_u$$

and since the next pass $(p + 1)$ adopts exactly these two values as the new filters:

$$f_{\min}^{(p+1)} = x_v, \qquad f_{\max}^{(p+1)} = x_u$$

Invariant 3.1 still holds at the end of the pass.

Termination and final selection: Finally, it remains to verify that each pass actually reduces the upper bound on the number of candidates. From the choice of the new filters it follows that, in the next pass, the actual number of candidates n_{p+1} remains bounded by the new upper bound $\bar{n}_{p+1}$ returned by PASS:

$$n_{p+1} \leq \bar{n}_{p+1} = (2r_p - 1)2^{r_p}$$

which, with the choices of s and r fixed in Section 3.3.2 (denoting with subscript p the corresponding pass), becomes:

$$\bar{n}_{p+1} = O\left(\frac{\bar{n}_p(\log \bar{n}_p)^2}{S}\right)$$

For S sufficiently large with respect to $(\log N)^2$, this relation guarantees that the upper bound on the number of candidates decreases strictly pass after pass, until this bound "collapses" to $S - 2$ or less (in a finite number of passes); this is the moment at which, by construction of MP-SELECT, the main loop stops.

At that point all the remaining candidates can be kept in memory together with the two filters, without it being necessary to build a further sample. By Invariant 3.1, the element $a_{(k)}$ can then fall into one of these two cases: either it coincides with one of the filters or it belongs to the set C collected during the last pass. In the first case it is trivially identified through its rank. In the second, it is enough to sort C and select the element of relative rank k_C, obtaining exactly $a_{(k)}$.

Under the assumptions considered, the algorithm therefore terminates after a finite number of passes and correctly returns the element of rank k.

Chapter 4

Implementation and experimental analysis

4.1 Implementation choices

The algorithm was implemented in *Python* [11], mainly for two reasons. The first is that it makes it possible to write "clean" code very close to the pseudocode of the previous chapter, and therefore easy to compare with this document. The second is the wide availability of libraries for data analysis, which makes it easy to obtain clear plots; for this work *Matplotlib* [12] was used.

The input tape is represented by a text stream containing one integer per line. The elements are read one at a time, in the order in which they arrive, and it is never possible to directly access an arbitrary position of the sequence. To start a new pass the "read head" is brought back to the beginning, simulating precisely the rewinding of the tape as in the theoretical model. In this way each pass of the implementation corresponds to a complete scan of the tape, and the count of passes will therefore coincide with the number of times the tape is reset.

Memory, on the other hand, requires more care. The parameter S denotes the number of logical cells available to the algorithm, that is, how many elements can be kept at the same time, and not the RAM occupied by the *Python* interpreter. The latter is much larger, both because each object occupies many more bytes than a single cell and because the program must also handle service data that are not elements of the input (for example: $q_{\max}$ and $q_{\min}$, the reference to the tape, the FOUND or CONTINUE label returned by each pass, and the parameters s, r, k and $\bar{n}$). It was therefore decided to impose the constraint only on the model and not on the whole process, so as to avoid complications that are not the subject of this study. During a pass the implementation keeps in memory only the

working sample of s elements, one subsample of $s/2$ elements for each of the r levels, and the two filters. These structures are created only once, already with the expected size, and *thinning* and *merge* work directly *in-place*, without creating copies. Moreover, before each pass it is checked that the total space does not exceed S.

As input, random permutations of the integers from 1 to N were considered, written one per line on the tape. The algorithm actually does not depend on the type of the values: it only requires them to be distinct and comparable with each other, so it would work in the same way with real numbers. Integers, however, are the most convenient choice for experimentation, because they make it possible to generate, simply and reproducibly, sequences of any size in which the distinctness of the elements is guaranteed by construction. With floating-point values, instead, an additional check would be needed to avoid duplicates, for example due to rounding.

Finally, the meaning of S also needs to be clarified. The memory of the model is not measured in "commercial" units, such as bytes, megabytes or gigabytes, but in a "positional" way: a cell contains exactly one element of the input, and S therefore indicates how many elements can be kept at the same time. The input size N and the memory S are thus expressed in the same unit, the number of elements, which for the input corresponds to the number of lines of the file. As long as the elements belong to *built-in* types of bounded size, such as integers or real numbers, the fact that a value occupies 2, 4 or 8 bytes changes the physical memory only by a constant factor and does not alter the ratio between S and N. The situation would be different if the elements were objects of variable or very large size, such as long strings or complex records: in that case a single cell could require an uncontrolled amount of memory, and the definition of S would need to be revised. For the inputs considered in this thesis, however, the positional measure correctly describes the resources used.

4.2 Implementation of the algorithm

The code developed for this work is collected in a public GitHub repository, so that it can be consulted and the experiments presented in this chapter can be reproduced. The repository is available at the following address:

https://github.com/ill-barba/bsc-thesis-unipd-median-selection



and is organised in three parts: the source code with the algorithm, the tests to verify its robustness, and finally the files to run the experiments and produce the plots. The *tree* of the repository is shown below, with a short description of each file:

```
bsc-thesis-unipd-median-selection/
|-- src/
|   `-- multipass.py              MP-SELECT and helper routines
|-- tests/
|   `-- test_multipass.py         tests of the code
|-- experiments/
|   |-- metrics_multipass.py      counting passes and candidates
|   |-- experiment_passes.py      experiment on passes
|   |-- experiment_candidates.py  experiment on candidates
|   `-- results/                  CSV data and plots
|-- thesis-alessandro-barbacetto.pdf
|-- README.md
|-- LICENSE
`-- NOTICE
```

The code described in Chapter 3 is all in `multipass.py`, which does not depend in any way on the rest of the project. The files in the `experiments/` folder use it by importing it from outside: they generate the inputs, run the algorithm and measure its behaviour, saving the results in `results/`. The `README.md` file explains how to install the dependencies and run the tests and experiments, while the behaviour of the individual functions is documented directly in the code through *docstrings*. The software is distributed under the *Apache 2.0* license, reported in the `LICENSE` file, and the `NOTICE` file states who developed the project. Finally, a copy of this work is also available in the main folder.

4.3 Experimental methodology

The experiments aim to verify and graphically observe whether the behaviour of MP-SELECT is in line with the guarantees derived in Chapter 3. The theoretical analysis had provided two upper bounds: the maximum number of passes $P_{\max}$, which depends only on N and S, and the maximum number of candidates remaining after each pass. This experimental part therefore aims to ascertain that the implementation respects these guarantees and to observe where the actually measured values lie.

The inputs consist of the integers from 1 to N. A random permutation of this sequence is used, created by a random generator initialised with a fixed *seed*; for a given seed the permutation is deterministic, and every run can be repeated identically. In addition, sequences sorted in increasing and decreasing order were also considered, which represent two limit cases of the arrangement of the data on the tape.

The measurements are performed by the program `metrics_multipass.py`, separate from the main algorithm `multipass.py`. The number of passes P is obtained by counting the rewinds of the tape, while, to observe the candidates, the process executing a single pass is "wrapped" from the outside, so as to record its state and the new filters. The actual number of candidates is then determined by examining a complete copy of the input, kept only for the measurements and therefore not subject to the constraint of the S memory cells of the algorithm. Finally, the same module computes the expected theoretical bounds by applying the recurrence on $\bar{n}$, without examining any data.

The observed quantities are the memory S, which is the parameter being varied, the number of passes P, compared with $P_{\max}$, and the candidates. For the latter, n_p denotes the number of elements between the filters produced by pass p and $\bar{n}_p$ the theoretical bound, setting initially by convention $n_0 = \bar{n}_0 = N$.

Finally, note that the input contains exactly the integers from 1 to N, so the element of rank k equals $N + 1 - k$ whatever the order on the tape, and the result of MP-SELECT is compared with this value to assess its correctness. It is also verified that the count of passes is consistent with those performed and that after each pass $n_p \leq \bar{n}_p$ holds. On these bases, two "experiments" were carried out, which are discussed in the next sections.

4.4 Memory and number of passes

The first analysis concerns the main *trade-off* of the Munro and Paterson model, that is, how many times the tape must be scanned (the number of passes) as a function of the available memory. Four input sizes, each ten times larger than the previous one, were considered, namely $N = 10^3, 10^4, 10^5, 10^6$, looking in each case for the median $k = \lceil N/2 \rceil$ on a random permutation (with seed 2026).

For each of the four values of N, the memory S was not varied over all integer values, but over a geometric grid, in which each value is obtained by multiplying the previous one by $2^{1/4} \simeq 1.189$, that is, increasing it by about 19% (rounding the result to the nearest integer). Therefore, after four increments the memory has doubled. With this choice the increments are small in absolute value when the memory is small and become progressively larger as S grows. In this way it is possible to observe, with a limited number of runs, a very wide memory range, going from a few hundred cells up to the whole input.

The starting point of the horizontal axis depends on the fixed input size N and is denoted by $S_{\min}(N)$: it is the smallest integer memory value for which the recurrence of the algorithm manages to reduce the bound on the candidates down to the final selection, for that input size. For the four values of N considered, the key constraint concerns the sec-

ond pass: if the sample is not large enough to decrease the number of levels with respect to the previous one, the bound remains unchanged and the algorithm makes no progress. Denoting by r_1 the number of levels of the first pass and by $\bar{n}_1 = (2r_1 - 1)2^{r_1}$ the bound it returns, one finds:

$$S_{\min}(N) = \left\lceil 2(2r_1 - 1)\log_2 \bar{n}_1 \right\rceil,$$

from which it can be computed explicitly for each N:

- $N = 10^3$: $S_{\min} = \lceil 22 \log_2 704 \rceil = 209$;
- $N = 10^4$: $S_{\min} = \lceil 34 \log_2 8704 \rceil = 445$;
- $N = 10^5$: $S_{\min} = \lceil 46 \log_2 94208 \rceil = 761$;
- $N = 10^6$: $S_{\min} = \lceil 58 \log_2 950272 \rceil = 1152$.

These values correspond "by eye" to the starting points of the various curves in Figure 4.1. Starting from these values, the memory then grows according to the increment:

$$S_j = \mathrm{round}\Big(S_{\min}(N) \cdot 2^{j/4}\Big), \qquad j = 0, 1, 2, \ldots$$

as long as $S_j < N + 2$. When $S = N + 2$ is finally reached, the memory is sufficient to hold the whole input together with the two filters and the algorithm terminates.

Therefore, in the plot the horizontal axis S is on a logarithmic scale. The memory indeed varies over almost four orders of magnitude and, if a linear axis had been adopted, the initial part of the curves, where the number of passes changes most rapidly, would have been "squeezed" towards the origin. With a logarithmic scale, on the contrary, values that always grow by the same percentage turn out to be equally spaced.

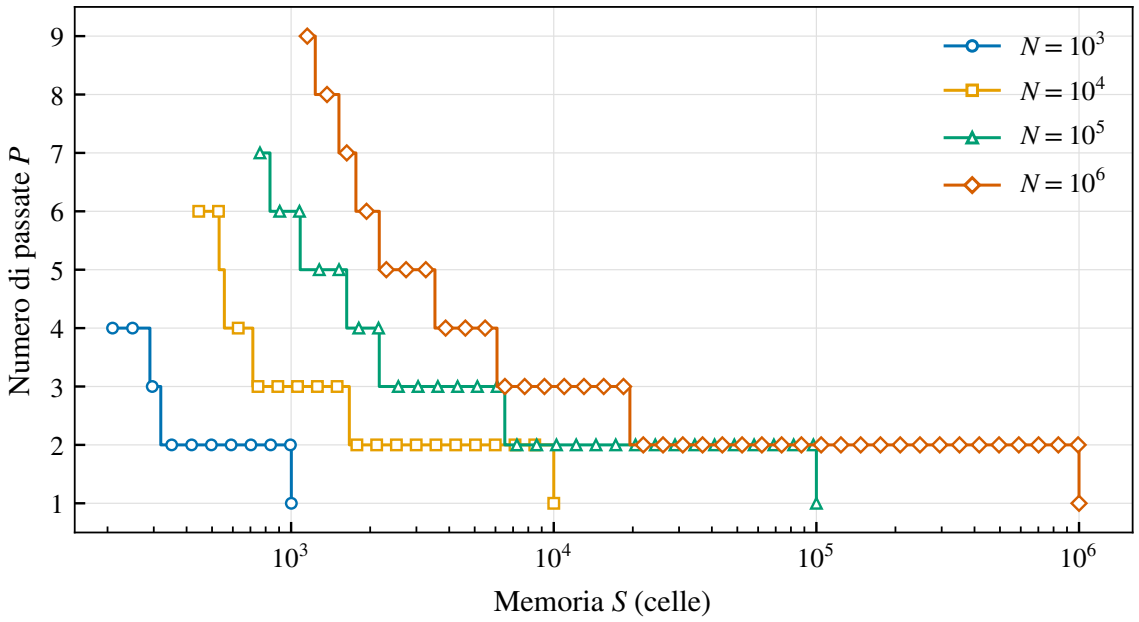


Figure 4.1: Number of passes as a function of the memory S (logarithmic scale), for $N = 10^3, 10^4, 10^5, 10^6$ and $k = \lceil N/2 \rceil$. The "step" curves represent $P_{\max}$, computed from the recurrence on $\bar{n}$, while the hollow markers are the observed values of P.

In Figure 4.1 the first thing one notices is that all the markers fall exactly on their respective "step" curves, so the number of passes measured by running the algorithm always coincides with the one predicted by the recurrence. One aspect, however, should be noted: $P_{\max}$ is the maximum number of passes predicted by the theory, and P can therefore be smaller only if a pass ends early by returning FOUND. The fact that no marker falls below its line indicates that, on the inputs of interest, this never happened. It follows that, in the absence of early terminations, the number of passes is fixed by the parameters of the recurrence.

As for the behaviour of the plot, it is overall the expected one. Indeed, no curve "rises again", that is, as the memory increases the number of passes never grows, but decreases in a discrete way, in steps. The reason for all this lies in the recurrence itself. The parameters s and r of each pass are integers, so a small increase of S can leave them unchanged, and with them the bound on the candidates. Even when the bound of a pass decreases, the overall number of passes decreases only if the reduction is enough to bring $\bar{n}$ below $S - 2$ one pass earlier. As a result, $P_{\max}$ remains constant over whole intervals of memory and then drops at certain thresholds, always by a single unit at a time.

More interesting, instead, is the width of the steps, which on a logarithmic scale indicates by how many times the memory must grow to save one pass. The steps are generally "narrower" near the minimum memory $S_{\min}(N)$ and "widen" rapidly moving to the right, up to the two-pass step ($P = 2$), which alone occupies a large part of the plot. This

behaviour is explained by the reduction guaranteed at each pass, derived in Section 3.5, which divides the bound on the candidates by a factor at least of the order of $S/(\log N)^2$; after the sampling passes, the bound must have dropped enough to be held in memory. It follows that, up to logarithmic factors, the memory needed to finish in P passes is of the order of $N^{1/P}$, which becomes $\sqrt{N}$ in the case of interest with two passes, the cube root for three, and so on [3]. When the passes are many (larger values of N), successive roots of N are close to each other and a small increase in memory is enough to save one; when they are few, instead, they are very far apart, and between $\sqrt{N}$ and N itself the gap is huge.

The comparison between the four curves also shows how little the algorithm is affected by the input size. The minimum memory grows much more slowly than N: for one million elements a little more than a thousand cells are enough, about one thousandth of the input, at the price of nine passes. Similarly, for the same memory, an input ten times larger requires only a few more passes, and when the memory is large enough the difference disappears altogether: between about 20000 and 100000 cells the curves for $N = 10^5$ and $N = 10^6$ overlap on the two-pass step, even though they refer to inputs differing by a factor of ten.

Finally, since $P_{\max}$ depends only on N and S, the plot can be read in both directions. Given the available memory, it indicates how many passes will be needed and, conversely, given the number of passes one is willing to perform, it indicates how much memory is needed. In both cases the answer is known even before reading and analysing the data, thanks to the deterministic guarantee derived in Chapter 3, which is independent of the order of the elements and of the rank being sought.

4.5 Actual candidates and theoretical bound

In the previous section the number of passes is determined by the recurrence on $\bar{n}$, which does not depend on the data, but this does not say how many candidates actually remain between the filters after each pass; it only provides a bound on them. This second analysis focuses precisely on this. For each pass p, the actual number of remaining candidates n_p, counted directly on the input, is compared with the corresponding theoretical bound $\bar{n}_p$, which depends only on the number of levels r_p of the sample built in that pass. The theory guarantees that $n_p \leq \bar{n}_p$ always holds, but does not say how far apart the two values are.

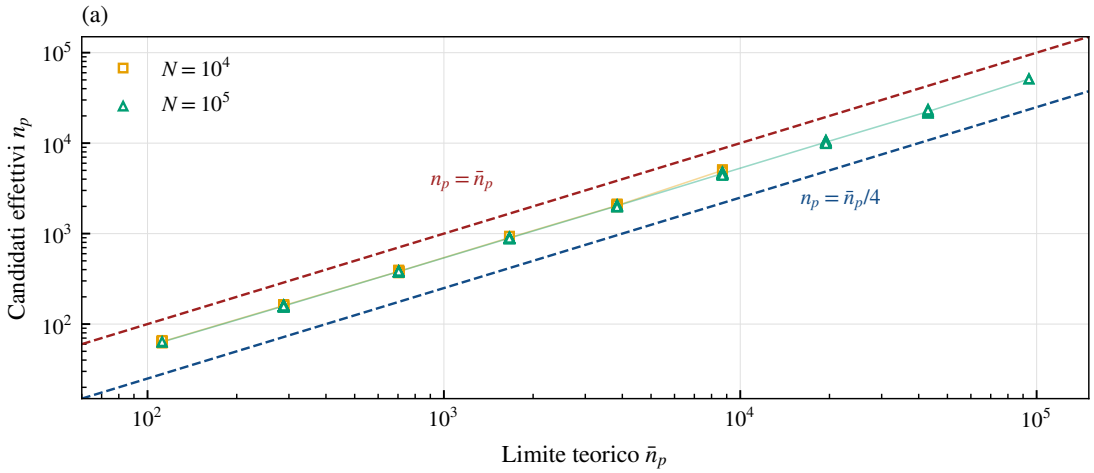
The comparison is interesting because it is precisely the bound, and not the actual number of candidates, that drives the algorithm: indeed, the parameters of the next pass and the moment at which the final selection starts depend on $\bar{n}_p$. For this reason the bound is very conservative, and its value is obtained by assuming that the two new filters simultane-

ously occupy the most unfavourable position allowed by Definition 3.9, that is, the highest possible one for the upper filter and the lowest one for the lower filter. This is because the bound must hold for any input, and the analysis serves to understand how pessimistic it turns out to be and what this implies for the behaviour of the algorithm.

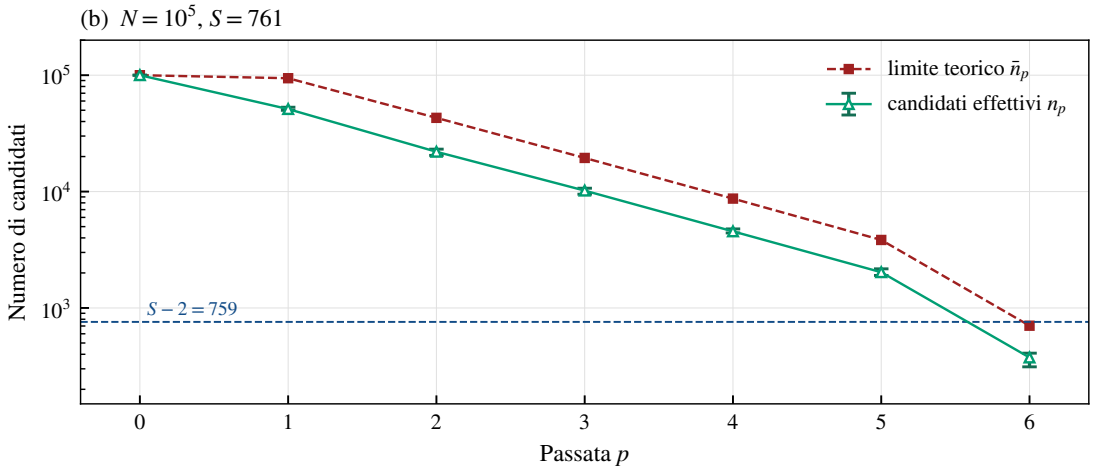
For this second experiment two input sizes were considered, namely $N = 10^4$ and $N = 10^5$, looking in both cases for the median $k = \lceil N/2 \rceil$. Unlike before, for each configuration 30 random permutations were used, generated with seeds from 1 to 30, for simplicity. As a result, the theoretical bound $\bar{n}_p$ remains unchanged for given N and S, as mentioned above, whereas the actual number of candidates n_p can change depending on the arrangement of the elements on the tape. The different permutations therefore make it possible to observe both its mean value and its variability. In this case too, the memory S was varied over a geometric grid, starting from the values $S_{\min}(N)$ already derived in the previous section. The grid continues up to the last value of S smaller than $N + 2$, the threshold at which the memory becomes sufficient to hold the whole input together with the two filters.

In the first panel (a), Figure 4.2a, the results of all the configurations considered are collected and averaged. For each pass p, the theoretical bound $\bar{n}_p$ is reported on the horizontal axis and the mean of the actual candidates n_p over the 30 permutations on the vertical axis. Each hollow marker therefore represents a triple (N, S, p). Both axes are on a logarithmic scale, since the number of candidates varies over several orders of magnitude. Two dashed lines are also shown, providing a visual "guide". The red one represents the theoretical bound $n_p = \bar{n}_p$, while the blue one is $n_p = \bar{n}_p/4$; it will be observed later that the interpolation of the markers would lie exactly on the guide $n_p = \bar{n}_p/2$.

The second panel (b), Figure 4.2b, considers instead a single configuration, with an input of size $N = 10^5$ and a memory $S = 761$, again chosen by looking at $S_{\min}(N)$. In this case the horizontal axis represents the index of the current pass p, while the vertical one, again on a logarithmic scale, indicates the resulting number of candidates. Here the theoretical bound $\bar{n}_p$, represented by the dashed red curve, is compared with the mean of the actual candidates n_p over the 30 permutations, represented by the green curve. The point $p = 0$ represents the initial situation, in which $n_0 = \bar{n}_0 = N$, while the blue horizontal line indicates the threshold $S - 2$, below which it is possible to proceed with the final selection.



(a) Mean of n_p as a function of $\bar{n}_p$ for each triple (N, S, p), on a logarithmic scale. The dashed lines represent the bound $n_p = \bar{n}_p$ in red and the reference $n_p = \bar{n}_p/4$ in blue.



(b) Bound $\bar{n}_p$ and mean of n_p, with bars between minimum and maximum, pass by pass for $N = 10^5$ and $S = 761$. The dashed line indicates the threshold $S - 2$.

Figure 4.2

Looking at Figure 4.2a, one notices that the points are arranged in vertical columns, each associated with a different number of levels r_p of the sample. This happens because the theoretical bound $\bar{n}_p$ depends only on r_p, so configurations with different memory values and passes can share the same bound.

Looking at Figure 4.2b, one notices that the actual number of candidates decreases progressively together with the theoretical bound, always remaining below it, except in the initial situation in which the two values coincide. The "bars" representing the minimum and maximum over the permutations also remain within the bound, showing that the guarantee is respected not only by the mean, but by all the runs considered.

The first result, observable in Figure 4.2a, is guaranteed by the theory: the bound is always respected, both by the means and by the individual runs. The margin is also very wide, because in the sampling passes not even the most unfavourable observed case reaches three quarters of $\bar{n}_p$. More interesting is the distribution of the points: over about three orders of magnitude, they always seem to lie roughly halfway between the two dashed guide lines. It can therefore be concluded that the actual candidates are, on average, a little more than half of the theoretical bound.

Even more regular is the behaviour within the "imaginary columns" mentioned above. Indeed, the points obtained with different memories, in different passes and for both input sizes lie practically at the same height. For the same number of levels, the first and the last pass, with little or much memory, leave on average the same fraction of candidates. A possible intuitive explanation of this behaviour comes from the way the bound is defined. It includes both the distance that normally separates the two filters and the margin needed to cover their most unfavourable positions, two contributions of similar size.

Other simulations, not shown in the plots, make it possible to also consider other ranks and sorted inputs. The bound is always respected, but the margin changes. The rank $k = \lceil N/10 \rceil$ behaves similarly to the median, while for other "extreme" ranks the candidates are considerably fewer, especially for $k = 1$. In these cases one observes that one of the two filters tends to remain fixed at a sentinel value, for example infinity (for $k = 1$, indeed, the upper filter is never updated). This could explain the smaller number of candidates observed: the search interval is in fact bounded on one side only, while the theoretical bound is computed considering the most unfavourable case for both filters. Finally, on already sorted inputs some early terminations were observed, for example a run with $k = N$ that ended with FOUND, and therefore with a number of passes smaller than $P_{\max}$.

Moving to Figure 4.2b, one can observe how the margin between actual candidates and theoretical bound develops over the individual passes. The distance between the two curves opens up from the first pass and then remains almost constant. The actual candidates,

instead, already drop to a little more than half of the bound, that is, to about half of the input. The initial guarantee is therefore particularly pessimistic, but this is related to the minimum memory chosen. Indeed, in configurations with more memory, the first pass already significantly reduces the theoretical bound as well.

The bars on the markers of panel (b) show instead how much the individual permutations deviate from the mean: they are barely visible in the first passes and widen especially in the last one, when the candidates are by then only a few hundred. It is reasonable to think of an averaging effect. Indeed, over many elements the irregularities of a single permutation tend to compensate each other, while over a few hundred candidates even a few elements more or less weigh more heavily on the overall mean. It can therefore be deduced that the number of candidates remaining after a pass is more stable the larger the bound was.

Another interesting aspect of Figure 4.2b concerns the threshold $S - 2$. The algorithm continues sampling as long as $\bar{n}_p > S - 2$; it is therefore the bound, and not the actual number of candidates, that decides the transition to the final selection. In the figure the two curves cross the threshold in the same pass, but this is not always the case in general. When the bound of the second-to-last pass exceeds the threshold, the actual candidates may already be below it one pass earlier, thus resulting in a "slowdown". Whether this happens or not depends, as for the "steps" of Section 4.4, on where the threshold falls with respect to the discrete values that the bound can take. In these cases the additional pass is due only to the caution of the bound, and its cost in any case corresponds to a complete scan of the tape.

Another consequence of this margin concerns the memory used in the final selection. Since the bound often ends well below the threshold $S - 2$ and rarely coincides with it, and the actual number of candidates is in turn smaller than the bound, part of the available cells remains unused. In the case of Figure 4.2b, the candidates occupy about half of the available cells; the memory reserved for the final pass is therefore sized for the worst case and, in the observed runs, remains largely empty.

The algorithm, on the other hand, could not base its decisions directly on the actual number of candidates, because at the end of a pass it knows the new filters, but not how many elements lie between them, and to know this it would have to scan the tape again. The bound, instead, is known in advance and deterministically guarantees that the memory constraint is respected.

4.6 Final remarks and limitations

Taken together, the two experiments show two sides of the same choice. MP-SELECT uses the actual data to choose the new filters, but the parameters of each sample and the moment at which sampling stops depend only on the bound $\bar{n}$. The only exception is the case, rarely observed, in which a filter coincides with the element being sought. On the one hand, this makes the maximum number of passes predictable, following the theoretical recurrence and decreasing in "steps" as the memory increases. On the other hand, the same bound turns out to be very conservative: in the configurations considered it overestimates the actual candidates and in some cases forces the algorithm to perform a pass that the data alone would not have required. Predictability and caution are therefore, respectively, the advantage and the drawback of the same guarantee.

The study also has some limitations. The inputs are synthetic, random permutations and sorted sequences of distinct integers, while real data may have different distributions and sometimes contain repeated values, which the model excludes by assumption. Memory, moreover, is measured in logical cells, according to the model described in Section 4.1, and does not coincide with the RAM actually occupied by *Python*. Finally, the analysis measures the cost of the algorithm in number of passes, as in the theoretical model, and not in execution time, that is, how much a pass actually costs, which indeed depends on the medium on which the data are stored and on the hardware that determines how fast they can be reread.

A possible point of reflection for future development concerns precisely the margin observed between the theoretical bound and the actual number of candidates, which in the final selection often leaves part of the available memory unused. One could evaluate a heuristic variant of the algorithm that, when $\bar{n}$ is slightly above the threshold $S - 2$, tries to "anticipate" the final selection based on an estimate of the actual number of candidates. During this scan the elements would be collected in memory only up to the allowed capacity and, if the candidates were actually at most $S-2$, the selection could end without further sampling. Otherwise, it would be necessary to resume the ordinary procedure with a subsequent pass. A strategy of this kind would preserve the memory constraint, but would introduce the risk of performing an additional scan without any benefit, wasting execution time. It would therefore be interesting to study under which conditions the attempt is worthwhile, comparing the number of passes saved with those possibly added, in relation to the "empty" checks when the remaining candidates do not "fit" the available memory. The goal would not necessarily be to use all the available cells, but to make the best use of them.

It is precisely this last aspect that the next chapter emphasises. Indeed, in Chapter 5 the

algorithm is placed in a real scenario, in which data are acquired sequentially and recorded on a memory that can be reread several times, while the working memory is small and the time available for processing is limited, and respecting it is crucial.

Chapter 5

Application example

5.1 Application scenario

As an application of the multi-pass algorithm by Munro and Paterson, we consider an embedded system installed on a Formula 1 car during a qualifying session. The car is performing one or more preparation laps with the goal of bringing the tyres into the correct thermal window, so as to have maximum grip for a fast lap [5].

During each lap the device acquires the tyre temperature at high frequency and progressively records it on a non-volatile memory. The goal of the system is to establish, before the next crossing of the finish line, whether the thermal conditions allow the following lap to be driven as a *push lap*, or whether a further preparation lap is needed.

To keep the miniworld consistent, the following assumptions are made:

1. The four tyres show equivalent thermal behaviour, so there will be no wheels with different temperatures and therefore a single sequence of temperatures will be used.

2. For each preparation lap the temperature acquisition window starts when crossing the finish line and ends upon entering the last microsector of the third sector.

3. At the end of the acquisition window the sequence used for the decision is complete; any samples acquired afterwards do not belong to the sequence analysed for that lap.

4. The non-volatile memory is large enough to contain all the acquired samples, while the RAM available to the embedded system is much smaller and is modelled through the S memory cells (of Chapter 3); the recorded data can be reread several times sequentially.

5. There exists a reference thermal window $[T_{\min}, T_{\max}]$ within which the tyre is considered to be in a favourable condition to start a fast lap.

6. It is assumed that, in the sequence of preparation laps, there always exists a lap whose median belongs to the window $[T_{\min}, T_{\max}]$, thus excluding the case in which the window is "skipped" between two consecutive laps without either of the two medians detecting it.

7. In accordance with the theoretical model adopted in Chapter 3, the samples are treated as distinct elements. Any coinciding temperature measurements can be distinguished through their acquisition order.

8. The computation of the median and the subsequent communication to the driver must be completed while driving through the last microsector, that is, before the start of the next lap.

From assumptions 2 and 3, acquisition and processing therefore take place at two distinct moments: during the first part of the lap the system only measures and stores the temperatures, while upon entering the last microsector the acquisition stops and the sequence just collected becomes the input of the multi-pass algorithm, which is run on it before the start of the next lap. At the end of the analysis the system communicates a binary indication to the driver:

- PUSH, if the next lap can be driven as a fast lap
- PREP, if instead a further preparation lap is needed

In this scenario the non-volatile memory plays the role of the input tape of the computational model, while the available RAM corresponds to the S working memory cells.

Remark 5.1 (Real time window)**.** To give an order of magnitude of the time available under assumption 8, one can take as a reference the pole position lap set by Charles Leclerc at Monza in 2022, completed in 1:20.161, with the third sector covered in about 26.553 seconds [8, 9].

Assuming that the third sector is divided into seven microsectors and considering, for simplicity, their average duration, one obtains:

$$\frac{26.553}{7} \approx 3.79 \text{ s}$$

The last microsector is here ideally taken to start in the area of the Alboreto corner and to end at the finish line; the value of 3.79 seconds gives an order of magnitude of the available time window.

The operations performed on the samples already in RAM, such as comparisons, *thinning* and *merge*, take place on time scales much shorter than a second. Moreover, the

overall execution of the algorithm also includes several sequential rereads of the memory, and its actual time should therefore be verified experimentally. In this miniworld, however, it is assumed, by Assumption 8, that a window of the order of 3–4 seconds is amply sufficient to complete the analysis and communicate the decision to the driver.

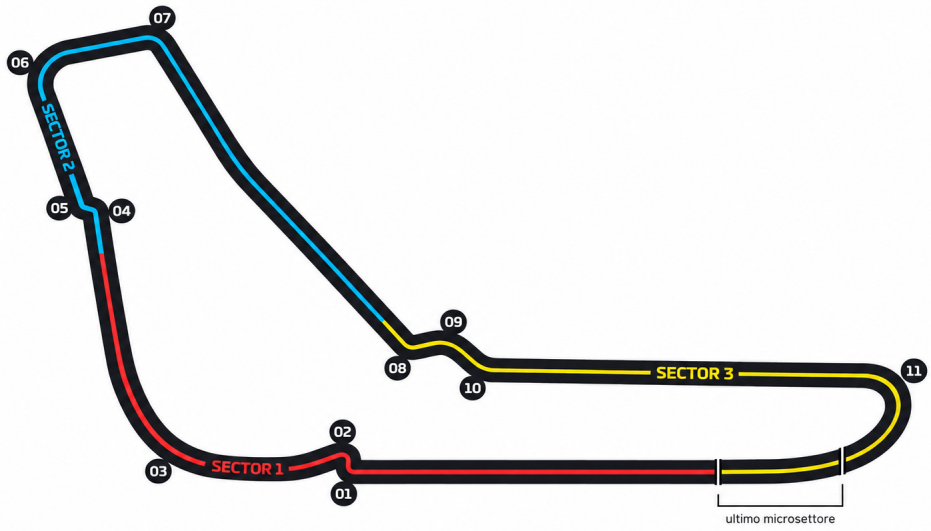


Figure 5.1: Simplified layout of the Monza circuit [10], highlighting the stretch of the last microsector. The temperature acquisition window ends at this portion, where processing begins.

5.2 Simplified model

Remaining faithful to assumption 1, the system is therefore represented with a single sequence of temperatures for each preparation lap. Let:

$$T_j = (t_{j,1}, t_{j,2}, \ldots, t_{j,N_j})$$

be the sequence of temperature samples acquired during lap j, where N_j is the number of measurements taken in the window defined by assumption 2 in lap j, that is, from the crossing of the finish line to the entry into the last microsector of the third sector.

Remark 5.2 (Samples per lap)**.** The number of samples N_j is not necessarily the same for all laps. Although the sampling frequency is always constant, the duration of the acquisi-

tion window can instead change from lap to lap, depending on the corresponding lap time. Consequently, the total number of measurements collected can also change.

Moreover, within the same window two samples may have the same temperature value. Such measurements are not removed, since this would alter the distribution of the data and therefore potentially also the median. In accordance with assumption 7, any coinciding values are instead considered distinct through their acquisition order.

The median is then computed on T_j: $m_j = \text{median}(T_j)$, which corresponds to a selection problem with target rank: $k_j = \left\lceil \frac{N_j}{2} \right\rceil$ solved directly with the multi-pass algorithm:

$$m_j = \text{MP-SELECT}(T_j, k_j, S)$$

Note that in this example the choice of the median (rather than an average of the samples) is not accidental: during a lap, short temperature variations or single anomalous samples can occur. These variations are mainly due, for example, to a momentary departure from the ideal racing line, to a local area of the asphalt that differs from the rest of the circuit, or to an isolated measurement error. Such values can significantly influence and "distort" the mean even if they concern only a small part of the measurements, while the median, depending mainly on the position of the sorted samples, provides an indicator that is more stable and robust to these changes.

5.3 Reference thermal window

The decision of the system is based on the comparison between m_j and the reference thermal window defined in assumption 5. The two endpoints $T_{\min}$ and $T_{\max}$ are not determined by the algorithm, but are application parameters provided as input to the system.

Their value depends on the characteristics of the tyre and on the operating conditions considered. In particular, the thermal window can vary depending on the compound used or, more generally, on the conditions affecting the interaction between tyre and track, for example the available grip level and the abrasive characteristics of the asphalt [5]. For this reason $T_{\min}$ and $T_{\max}$ must be established a priori on the basis of the experimental data available for the specific scenario considered.

Once the median m_j of the j-th lap has been computed, three cases are distinguished:

- if $m_j < T_{\min}$, the observed temperature is still below the reference window, so the system communicates PREP
- if $T_{\min} \leq m_j \leq T_{\max}$, the median of the lap falls within the window considered optimal, so the system communicates PUSH

- if $m_j > T_{\max}$, the observed temperature exceeds the reference window, so the system communicates PREP

In this model it is also assumed that, during a normal sequence of preparation laps, the tyre temperature increases progressively, or at most remains temporarily stable. Consequently, the medians m_j of successive laps also follow a non-decreasing trend. In other words, there are no sudden temperature spikes between consecutive laps, or abrupt "drops", that could make the system skip the optimal window and make it unreachable.

Formally, it is therefore also assumed that the increase between two consecutive laps is gradual enough not to allow a direct transition from a condition:

$$m_j < T_{\min}$$

to a condition:

$$m_{j+1} > T_{\max}$$

In other words, it is assumed that there cannot exist two consecutive laps such that $m_j < T_{\min}$ and $m_{j+1} > T_{\max}$.

In summary, this assumption describes the behaviour under normal preparation conditions. Anomalous events, such as going off track, an interrupted lap or the retirement of the car, are not considered in the base model.

5.4 Identifying the preparation laps

Given a sequence of preparation laps, the goal is to identify the first lap for which the favourable conditions for a fast lap are met. We therefore define:

$$j^* = \min \left\{ j \;\middle|\; T_{\min} \le m_j \le T_{\max} \right\}$$

The lap indexed by j^* is the first one at the end of which the system communicates PUSH, so the next lap can be driven as a possible *push lap*. For each lap j the decision is therefore obtained by applying the multi-pass algorithm to the sequence T_j to compute its median $m_j = \text{MP-SELECT}(T_j, k_j, S)$, and finally comparing it with the thermal window to determine whether to communicate PUSH or PREP.

Remark 5.3 (Existence of the optimal lap)**.** The existence of a lap j^* satisfying the previous definition does not constitute an absolute guarantee, but follows from the assumptions

of the simplified model. In particular, it is assumed that, under normal preparation conditions, the tyre temperature increases progressively and does not completely cross the window $[T_{\min}, T_{\max}]$ with a direct jump between two consecutive laps. Under this assumption, and by assumption 6, according to which there always exists a lap whose median belongs to the window, by continuing through the sequence of preparation laps one sooner or later reaches a condition for which: $T_{\min} \leq m_j \leq T_{\max}$ and the set over which the minimum in the definition of j^* is taken is therefore non-empty.

This, however, is a condition valid only in the ideal scenario considered here, and not a universal guarantee. In a real context, indeed, the sequence of preparation laps could be interrupted before reaching the desired window, for causes external to the model, for example: an accident leading to the retirement of the car, or the display of a yellow flag, a red flag or the safety car, interrupting or ending the qualifying session early. In such cases the sequence of preparation laps ends before the required condition is satisfied, and the lap j^* does not exist.

5.5 Possible extension to four tyres

The base model assumes, through assumption 1, that the four tyres have the same thermal behaviour, thus representing the system with a single sequence of temperatures. A more realistic version would instead keep the data coming from each tyre separate.

Let:

$$w \in \{FL, FR, RL, RR\}$$

denote the position of the tyre, where FL is the front left, FR the front right, RL the rear left and RR the rear right. For each lap j there are therefore four distinct sequences of samples, one for each tyre:

$$T_j^w = (t_{j,1}^w, t_{j,2}^w, \ldots, t_{j,N_j^w}^w) \qquad w \in \{FL, FR, RL, RR\}$$

By applying the multi-pass algorithm separately to the four sequences one obtains, for each lap, one median per tyre: m_j^{FL}, m_j^{FR}, m_j^{RL}, m_j^{RR}. The selection problem therefore remains unchanged, and simply has to be solved four times instead of once. An application problem arises, however: how to combine the four medians into a single PUSH or PREP decision.

Possible decision strategies. The most conservative criterion requires all four tyres to be in the optimal thermal window at the same time, and communicates PUSH only if:

$$T_{\min} \leq m_j^w \leq T_{\max} \qquad \forall w \in \{FL, FR, RL, RR\}$$

It is enough for a single one of the four medians to fall outside the window for the system to communicate PREP. The criterion therefore prevents an unfavourable condition on a single tyre from being compensated by the other three, but it is more restrictive.

A more permissive alternative consists in reducing the four medians to a single value by taking their mean:

$$\overline{m}_j = \frac{m_j^{FL} + m_j^{FR} + m_j^{RL} + m_j^{RR}}{4}$$

and communicating PUSH when $T_{\min} \leq \overline{m}_j \leq T_{\max}$. The limitation of this strategy is evident: very different temperatures can compensate each other, so that a tyre that is too hot and one that is too cold could produce a mean inside the window even if neither of them is actually in the desired condition.

A third option consists in weighting the four tyres differently according to the characteristics of the circuit and the loads they are subjected to, giving higher priority to some wheels rather than others. By introducing weights for each position $\alpha_{FL}, \alpha_{FR}, \alpha_{RL}, \alpha_{RR}$ such that:

$$\alpha_{FL} + \alpha_{FR} + \alpha_{RL} + \alpha_{RR} = 1$$

a weighted overall temperature can be computed:

$$\widetilde{m}_j = \alpha_{FL} m_j^{FL} + \alpha_{FR} m_j^{FR} + \alpha_{RL} m_j^{RL} + \alpha_{RR} m_j^{RR}$$

and PUSH is communicated when $T_{\min} \leq \widetilde{m}_j \leq T_{\max}$, giving greater weight to the tyres that, because of the layout of the circuit and the type of expected dynamic load, are more critical.

Of these three strategies, the last one is the closest to real tyre management, although in practice it does not translate into a simple weighted average: the two axles are monitored separately, seeking a thermal balance between front and rear also as a function of the car set-up and of the loads developed under braking, cornering and traction [5].

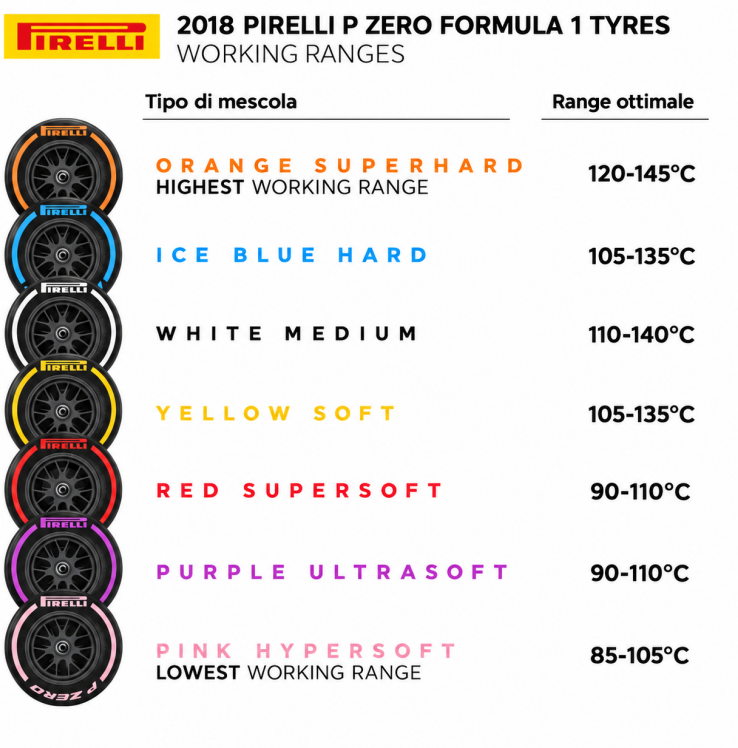


Figure 5.2: Working ranges of the Pirelli compounds used in Formula 1 in 2018 [6, 7].

Figure 5.2 illustrates a further simplification of the model: the values $T_{\min}$ and $T_{\max}$ indeed depend on the compound used, and in a real application they should be set according to the specific tyre. Not by chance, for a qualifying lap the choice almost always falls on the *hypersoft* compound, whose lower *working range* makes it possible to reach the optimal thermal window more quickly.

Remark 5.4 (Differences between the four tyres)**.** In the real scenario the four tyres generally do not follow the same thermal evolution. The differences derive first of all from the geometry of the circuit: corners taken mainly in one direction load some tyres more than others, while braking and traction stress the front and rear axles differently.

Driving style also introduces further differences: a lock-up under braking affects a specific tyre and can produce a *flat spot* or localised wear, just as going off the racing line or running over gravel alters its thermal evolution asymmetrically.

For this reason, in practice, the goal is not an average temperature of the car, but a thermal balance between the individual tyres and between the two axles. Assumption 1 therefore remains a simplification adopted to keep the base model compact.

Bibliography

[1] M. Blum, R. W. Floyd, V. Pratt, R. L. Rivest and R. E. Tarjan, *Time Bounds for Selection*, Journal of Computer and System Sciences, vol. 7, no. 4, pp. 448–461, 1973. https://doi.org/10.1016/S0022-0000(73)80033-9.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd edition, The MIT Press, 2009 (in particular the chapter on selection). https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/.

[3] J. I. Munro and M. S. Paterson, *Selection and Sorting with Limited Storage*, Theoretical Computer Science, vol. 12, no. 3, pp. 315–323, 1980. https://doi.org/10.1016/0304-3975(80)90061-4.

[4] C. Renzi, *Algoritmi Sequenziali e di Streaming per la Selezione della Mediana* (in Italian), report, 26 June 2026, material provided for consultation.

[5] Pirelli, *Seeing Through the Window*, 5 June 2017. https://www.pirelli.com/global/en-ww/race/racingspot/seeing-through-the-window-50931/.

[6] Pirelli, *Pirelli Reveals Its Rainbow Tyre Range for Formula 1*, 2017 (presentation of the 2018 range). https://www.pirelli.com/global/en-ww/race/racingspot/formula-1/pirelli-reveals-its-rainbow-tyre-range-for-formula-1-51271/.

[7] Motorsport.com, *Pirelli Could Widen Working Range of Tyres for 2020*, 2019. Also contains a comparison with the working ranges of the 2018 compounds. https://www.motorsport.com/f1/news/pirelli-widen-working-range-2020/4448199/.

[8] Formula 1, *Formula 1 Pirelli Gran Premio d'Italia 2022 – Qualifying*, official qualifying results of 10 September 2022. https://www.formula1.com/en/results/2022/races/1120/italy/qualifying.

[9] Fédération Internationale de l'Automobile (FIA), *Formula 1 Pirelli Gran Premio d'Italia 2022 – Qualifying Session Best Sector Times*, 10 September 2022. https://www.fia.com/sites/default/files/2022_16_ita_f1_q0_timing_qualifyingsessionbestsectortimes_v01.pdf.

[10] Fédération Internationale de l'Automobile (FIA), *2022 Monza Event – Circuit Map*, document no. 1, 8 September 2022. https://www.fia.com/sites/default/files/doc_1_-_2022_monza_event_-_circuit_map_-_monza_2022.pdf.

[11] Python Software Foundation, *Python Documentation*, official documentation. https://docs.python.org/3/.

[12] Matplotlib Development Team, *Matplotlib: Visualization with Python*, official documentation. https://matplotlib.org/stable/.

Acknowledgements

You are probably expecting the usual sweet, tear-jerking sentences which, as you well know, are not really my thing. And, to be honest, I would not even have been able to write them: I would probably have ended up copying them from somewhere :)

So I thought it would be nicer, and above all more spontaneous, to thank you in person, trying not to forget anyone.

This way, if nothing else, time will be able to carry away any possible embarrassment. Although I am fairly sure that some of you (no names mentioned) will carve it into their memory and will never miss a chance to remind me of it.

A bit like Plato with Socrates, going against his wishes. For my part, I at least hope to always remember these moments with a smile.

Finally, before the actual thanks, I would like to send a big hug to those who have been closest to me on this journey: my parents and my grandparents. And also to those who have been *even too close* to me: to all the flatmates and refugees of *Casa Limentani*, past, present and future.